Tugas Mandiri

Implementasi End-to-End Distributed Tracing pada Pipeline Komunikasi Asinkron (Message Broker)

1. Tujuan Pembelajaran

Setelah menyelesaikan penugasan ini, mahasiswa diharapkan mampu:

- **Memahami prinsip *distributed tracing*** dan pelacakan alur eksekusi terdistribusi melalui skema *Trace ID* dan *Span ID* pada arsitektur komunikasi *asynchronous* berbasis *message broker* (Message Queue / Pub-Sub).
- **Mengimplementasikan *context propagation*** secara mandiri sesuai standar *W3C TraceContext specification* menggunakan pustaka OpenTelemetry pada protokol komunikasi AMQP (RabbitMQ) atau Kafka Record Headers.
- **Memvisualisasikan dan menganalisis hirarki *parent-child span*** secara utuh mulai dari *HTTP Request* masuk hingga pemrosesan pesan di sisi *Consumer* melalui *dashboard* Jaeger UI.
- **Menerapkan korelasi log terstruktur** dan penanganan *error* pada pemrosesan pesan asinkron untuk menjamin keterlacakan serta ketahanan sistem terdistribusi.

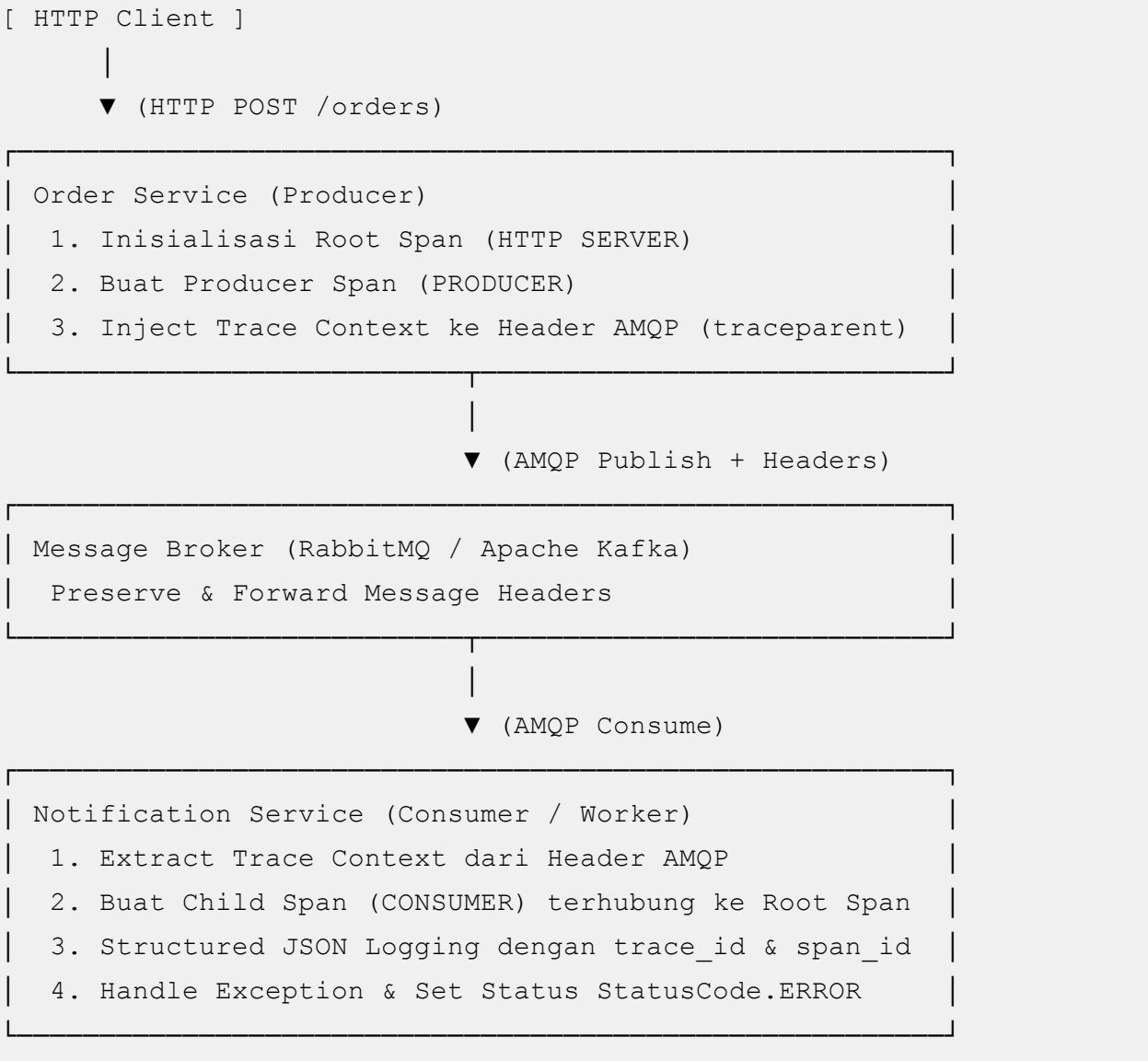
2. Skenario Sistem & Arsitektur

Dalam sistem terdistribusi berbasis *Event-Driven Architecture* (EDA), komunikasi *asynchronous* antar-layanan melalui *message broker* memutus terputus (*thread execution boundary*). Ketika *Producer* mempublikasikan pesan ke dalam antrean, thread eksekusi *Producer* langsung melepaskan memori dan menyelesaikan siklus *HTTP Request* (*non-blocking*). Di sisi lain, *Consumer* membaca dan memproses pesan tersebut pada thread, proses, atau lingkungan eksekusi yang terisolasi total serta berjalan di rentang waktu yang berbeda.

Pemutusan *thread boundary* ini menyebabkan alat pelacak otomatis bawaan protokol HTTP gagal meneruskan konteks lacakan secara implisit. Oleh karena itu, mahasiswa diwajibkan mengimplementasikan *manual context propagation* berbasis standar *W3C TraceContext specification* (traceparent dan tracestate). *Trace Context* harus *inject* ke dalam atribut *headers* pesan AMQP/Kafka di sisi *Producer* , lalu diekstrak

menggunakan API *TextMapPropagator* di sisi *Consumer* untuk merekonstruksi hierarki *parent-child span* secara menyeluruh.

Sistem yang akan dibangun terdiri dari 3 komponen utama dengan alur kerja terstruktur sebagai berikut:



Order Service (Producer / HTTP Entry Point):

- Menerima *HTTP Request* dari pengguna sebagai titik masuk transaksi.
- Menginisialisasi *Trace Context* utama dengan menciptakan *Trace ID* baru serta *Root Span* awal (berjenis SpanKind.SERVER atau SpanKind.CLIENT).

- *Inject Trace Context* aktif (traceparent yang berisi version-traceid-parentid-traceflags) ke dalam struktur metadata *Header* pesan AMQP (pada RabbitMQ) atau Kafka Record Headers menggunakan *TextMapPropagator* sebelum mempublikasikan pesan ke *Message Broker* .

Message Broker (RabbitMQ / Apache Kafka):

- Bertindak sebagai perantara (*broker*) komunikasi asinkron yang menerima pesan beserta metadata *Header* dari *Producer* .
- Meneruskan pesan tersebut ke antrean (*queue/topic*) tanpa merusak, memodifikasi, atau menghapus atribut *Trace Context* yang tersimpan di dalam *Header* .

Notification Service (Consumer / Worker):

- Membaca dan mengkonsumsi pesan dari antrian *Message Broker* .
- Mengekstrak (*extract*) *Trace Context* yang berada di dalam *Header* pesan AMQP/Kafka menggunakan *TextMapPropagator* .
- Membuat *Child Span* baru (berjenis SpanKind.CONSUMER) yang menetapkan *Trace Context* hasil ekstraksi sebagai *parent* , sehingga *Child Span* terhubung secara kontinu ke *Root Span* bawaan *Producer* .
- Mencatat log terstruktur (*structured JSON logging*) yang menyertakan atribut trace_id dan span_id secara otomatis guna keperluan korelasi log (*log correlation*).
- Menangani eksepsi/kesalahan pemrosesan dengan mencatat status *error* (StatusCode.ERROR) dan merekam rincian eksepsi pada *Span* sebelum memindahkan pesan gagal ke *Dead Letter Queue* (DLQ), lalu menyelesaikan (*end/finish*) *Span* .

3. Spesifikasi Teknis & Langkah Kerja

Mahasiswa bisa mengikuti panduan langkah kerja bertahap berikut dalam menyusun lingkungan pengembangan, mengimplementasikan kode program, serta memverifikasi hasil *tracing* .

3.1 Persiapan Environment (Docker Compose)

Siapkan docker dan sediakan file orkestrasi docker-compose.yml untuk menjalankan layanan infrastruktur *observability* (Jaeger) dan *message broker* (RabbitMQ).

```
version: '3.8'
services:
  jaeger:
    image: jaegertracing/all-in-one:latest
    container_name: jaeger-tracing
    ports:
      - "16686:16686" # Jaeger Web UI
      - "4317:4317"   # OTLP gRPC receiver
      - "4318:4318"   # OTLP HTTP receiver
    environment:
      - COLLECTOR_OTLP_ENABLED=true
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq-broker
    ports:
      - "5672:5672"   # AMQP protocol port
      - "15672:15672" # RabbitMQ Management Dashboard
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest
```

Jalankan infrastruktur pendukung dengan perintah:

```
docker compose up -d
```

3.2 Implementasi Kode Producer (Order Service)

Gunakan pustaka SDK OpenTelemetry sesuai bahasa pemrograman pilihan (contoh menggunakan Python).

requirements.txt :

```
opentelemetry-api
opentelemetry-sdk
opentelemetry-exporter-otlp-proto-grpc
pika
flask
```

Cuplikan Kode order_service.py (Producer):

```
import json
import pika
from flask import Flask, request, jsonify
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
OTLPSpanExporter
from opentelemetry.trace.propagation.tracecontext import
TraceContextTextMapPropagator
from opentelemetry.trace import SpanKind
# 1. Setup OpenTelemetry Tracer & OTLP Exporter ke Jaeger
provider = TracerProvider()
processor =
BatchSpanProcessor(OTLPSpanExporter(endpoint="localhost:4317",
insecure=True))
provider.add_span_processor(processor)
trace.set_tracer_provider(provider)
tracer = trace.get_tracer("order-service")
app = Flask(__name__)
def publish_to_rabbitmq(order_data, headers):
    connection =
pika.BlockingConnection(pika.ConnectionParameters('localhost', 5672))
    channel = connection.channel()
    channel.queue_declare(queue='order_notifications')
```

```
    # Publikasikan pesan beserta header AMQP yang memuat Trace
Context
    properties = pika.BasicProperties(headers=headers,
content_type='application/json')
    channel.basic_publish(
        exchange='',
        routing_key='order_notifications',
        body=json.dumps(order_data),
        properties=properties
    )
    connection.close()
@app.route('/orders', methods=['POST'])
def create_order():
    # 2. Inisialisasi Root Span untuk HTTP Handler
    with tracer.start_as_current_span("POST /orders",
kind=SpanKind.SERVER) as root_span:
        order_data = request.json or {"order_id": "ORD-9982", "item":
"Laptop", "amount": 15000000}
        root_span.set_attribute("order.id", order_data["order_id"])

        # 3. Buat Producer Span khusus u/ aksi publikasi ke Message
Broker
        with tracer.start_as_current_span("publish_order_created",
kind=SpanKind.PRODUCER) as pub_span:
            headers = {}
            # 4. Inject Trace Context aktif ke dalam dictionary
headers
            TraceContextTextMapPropagator().inject(carrier=headers)

            pub_span.set_attribute("messaging.system", "rabbitmq")
            pub_span.set_attribute("messaging.destination",
"order_notifications")

            publish_to_rabbitmq(order_data, headers)
```

```
        return jsonify({"status": "Order created & event published",
"order_id": order_data["order_id"]}), 202
if __name__ == '__main__':
    app.run(port=5000)
```

3.3 Implementasi Kode Consumer (Notification Service)

Implementasikan kode *Consumer* yang mengekstrak *Trace Context* , menangani eksepsi, dan mencetak *Structured Log* .

Cuplikan Kode notification_service.py (Consumer):

```
import json
import logging
import pika
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
OTLPSpanExporter
from opentelemetry.trace.propagation.tracecontext import
TraceContextTextMapPropagator
from opentelemetry.trace import SpanKind, StatusCode
# 1. Setup OpenTelemetry Tracer & OTLP Exporter
provider = TracerProvider()
processor =
BatchSpanProcessor(OTLPSpanExporter(endpoint="localhost:4317",
insecure=True))
provider.add_span_processor(processor)
trace.set_tracer_provider(provider)
tracer = trace.get_tracer("notification-service")
# Setup Structured JSON u/ Logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("notification-logger")
def log_structured(level, message, trace_id, span_id,
extra_data=None):
    log_payload = {
        "level": level,
```

```
        "message": message,
        "trace_id": f"{trace_id:032x}" if trace_id else None,
        "span_id": f"{span_id:016x}" if span_id else None,
        "data": extra_data or {}
    }
    print(json.dumps(log_payload))
def callback(ch, method, properties, body):
    # 2. Ambil headers dari pesan AMQP (carrier)
    headers = properties.headers if properties and properties.headers
else {}

    # 3. Extract Trace Context dari headers
    parent_context =
TraceContextTextMapPropagator().extract(carrier=headers)

    # 4. Buat Child Span terhubung langsung ke Root Context Producer
    with tracer.start_as_current_span("process_notification",
context=parent_context, kind=SpanKind.CONSUMER) as consumer_span:
        ctx = consumer_span.get_span_context()
        trace_id = ctx.trace_id
        span_id = ctx.span_id

        try:
            order_event = json.loads(body)
            consumer_span.set_attribute("order.id",
order_event.get("order_id"))

            # Simulasi pemrosesan notifikasi
            log_structured("INFO", "Notification processed
successfully", trace_id, span_id, order_event)
            ch.basic_ack(delivery_tag=method.delivery_tag)

        except Exception as e:
            # Penanganan Fault Tolerance & Error Status
            consumer_span.record_exception(e)
```

```
            consumer_span.set_status(StatusCode.ERROR,
description=str(e))
            log_structured("ERROR", f"Failed to process notification:
{str(e)}", trace_id, span_id)
            # Requeue atau lempar ke Dead Letter Queue (DLQ)
            ch.basic_nack(delivery_tag=method.delivery_tag,
requeue=False)
def start_consumer():
    connection =
pika.BlockingConnection(pika.ConnectionParameters('localhost', 5672))
    channel = connection.channel()
    channel.queue_declare(queue='order_notifications')
    channel.basic_consume(queue='order_notifications',
on_message_callback=callback)
    logger.info("Notification Service Consumer started. Waiting for
messages...")
    channel.start_consuming()
if __name__ == '__main__':
    start_consumer()
```

3.4 Validasi Tracing di Jaeger UI

1. Eksekusi *Order Service* dan *Notification Service* pada terminal terpisah.
2. Kirimkan beberapa *HTTP Request* pengujian menggunakan curl atau Postman:

```
curl -X POST http://localhost:5000/orders \
     -H "Content-Type: application/json" \
     -d '{"order_id": "ORD-1001", "item": "Buku Diktat Scalable",
"amount": 150000}'
```

3. Buka antarmuka Jaeger UI di browser pada alamat http://localhost:16686.
4. Pilih *Service* : order-service, lalu klik **Find Traces** .
5. Verifikasi hierarki *Span* yang wajib terbentuk dalam 1 *Trace* berkelanjutan:
    a. **Span 1 (Root Span):** POST /orders (*SpanKind:* SERVER, *Service:* order-service)
    b. **Span 2 (Child Span Producer):** publish_order_created (*SpanKind:* PRODUCER, *Service:* order-service)

c. **Span 3 (Child Span Consumer):** process_notification (*SpanKind:* CONSUMER, *Service:* notification-service)

6. Evaluasi *Tail Latency* (P95/P99) pada grafik *timeline* Jaeger UI untuk mengidentifikasi durasi waktu tunggu antrian (*queueing delay*) versus durasi eksekusi aktual di *Consumer* .
7. Cocokkan *Structured JSON Log* di terminal *Consumer* dan pastikan nilai atribut trace_id pada log persis sama dengan *Trace ID* yang tertera pada Jaeger UI.